Experiment 6:
Implement the POST and GET methods for a Student collection using Express.js and Mongoose to perform Create and Read operations. Return an appropriate message in the response.

1. **Install mongoose**

npm install express mongoose

2. **app.js**

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
// Middleware to parse JSON request data
app.use(express.json());
app.set('port', 3000);
// Connect to MongoDB
mongoose.connect('mongodb://127.0.0.1:27017/CollegeDB')
  .then(() => console.log('MongoDB connected'))
  .catch(err => console.log(err));
// Define Student schema
const studentSchema = new mongoose.Schema({
  name: String,
  usn: String,
  branch: String,
  semester: Number
});
// Create Student model
const Student = mongoose.model('Student', studentSchema);
// POST: Create a new student
```

```
app.post('/students', async (req, res) => {
  try {
    const student = new Student(req.body);
    await student.save();


    res.status(201).json({
      message: 'Student created successfully',
      student: student
    });
  } catch (err) {
    res.status(500).json({
      message: 'Error creating student',
      error: err.message
    });
  }
});
// GET: Read all students
app.get('/students', async (req, res) => {
  try {
    const students = await Student.find();
    res.status(200).json({
      message: 'Students retrieved successfully',
      students: students
    });
  } catch (err) {
    res.status(500).json({
      message: 'Error retrieving students',
      error: err.message
```

```
  });
 }
});
// Start server
app.listen(3000, () => { console.log('Server running on port 3000'); });
module.exports = app;
```

3. **Test the POST method**

   Send a POST request to `http://localhost:3000/students` using Postman.

   Request body — select Body → raw → JSON:

```
{
 "name": "Rahul",
 "usn": "4VV23IS001",
 "branch": "ISE",
 "semester": 5
}
```

4. **Test the GET method**

   Send a GET request to: http://localhost:3000/students

Note:

Method	Endpoint	Purpose	Mongoose operation
POST	/students	Creates a new student record	student.save()
GET	/students	Retrieves all student records	Student.find()

OUTPUT:

GET:

```
{
 "message": "Students retrieved successfully",
 "students": [
  {
   "_id": "generated_by_mongodb",
   "name": "Rahul",
   "usn": "4VV23IS001",
```

```
    "branch": "ISE",
    "semester": 5
   }
 ]
}
```


POST:


```
{
 "message": "Student created successfully",
 "student": {
   "name": "Rahul",
   "usn": "4VV23IS001",
   "branch": "ISE",
   "semester": 5,
   "_id": "generated_by_mongodb"
 }
}
```


Experiment 7:

1. app.js

```
const express = require('express');
const mongoose = require('mongoose');

const app = express();
app.use(express.json());

// Connect to MongoDB
mongoose.connect('mongodb://127.0.0.1:27017/CollegeDB')
   .then(() => console.log('MongoDB connected'))
   .catch(err => console.log(err));

// Define Student schema
const studentSchema = new mongoose.Schema({
   name: String,
   usn: String,
```

```
    branch: String,
    semester: Number
});

const Student = mongoose.model('Student', studentSchema);

// UPDATE: Update a student using USN
app.put('/students/:usn', async (req, res) => {
    try {
        const student = await Student.findOneAndUpdate(
            { usn: req.params.usn },
            req.body,
            { new: true, runValidators: true }
        );

        if (!student) {
            return res.status(404).json({
                message: 'Student not found'
            });
        }

        res.status(200).json({
            message: 'Student updated successfully',
            student: student
        });
    } catch (err) {
        res.status(500).json({
            message: 'Error updating student',
            error: err.message
        });
    }
});

// DELETE: Delete a student using USN
app.delete('/students/:usn', async (req, res) => {
    try {
        const student = await Student.findOneAndDelete({
            usn: req.params.usn
        });

        if (!student) {
            return res.status(404).json({
                message: 'Student not found'
            });
        }

        res.status(200).json({
            message: 'Student deleted successfully',
            student: student
        });
```

```
    } catch (err) {
        res.status(500).json({
            message: 'Error deleting student',
            error: err.message
        });
    }
});

// Start the server
app.listen(3000, () => {
    console.log('Server running on port 3000');
});
module.exports = app;
```

2. Test the output:

Create suitable records:

POST request
http://localhost:3000/students

```
{
  "name": "Rahul",
  "usn": "4VV23IS001",
  "branch": "ISE",
  "semester": 5,
  "_id": "generated_by_mongodb"
}

{
  "name": "xyz",
  "usn": "4VV23IS002",
  "branch": "ISE",
  "semester": 5,
  "_id": "generated_by_mongodb"
}
```

Test the UPDATE operation

PUT

http://localhost:3000/students/4VV23IS001

```
{
  "name": "Rahul Kumar",
  "branch": "CSE",
  "semester": 6
}
```

Test the DELETE operation

DELETE

http://localhost:3000/students/4VV23IS002

No request body is required.